
Résolution d'un problème inverse: déconvolution d'un signal triangulaire

Table of Contents

.....	1
Introduction	1
Construction du jeu de donnée	2
Simulation du filtre	3
Simulation du problème direct: première approche	4
Simulation du bruit de quantification	5
Calcul du bruit de quantification	6
Simulation du problème direct par approche matricielle	7
Calcul de la sortie non bruitée par approche matricielle	7
Déconvolution naïve dans le cas idéal (sans bruit)	9
Déconvolution naïve avec le signal bruité	10
Influence de la forme de la réponse impulsionnelle sur la reconstruction	11
Déconvolution par moindres carrés (ℓ_2) avec régularisation	13
Déconvolution par régularisation linéaire quadratique ($\ell_2\ell_2$)	16
Difficultés et Solutions	18
Conclusion	19
Références	19

Auteur: NAFIOU MAHAMANE Mahamadou Nazirou

Supervisé par: Mr Paul CATALA et Mr El-Hadi Djermoune

Université de Lorraine - Master2 ISC Pt RESET

Introduction

Dans de nombreuses applications du traitement du signal et de l'imagerie numérique, il est nécessaire de récupérer un signal original à partir d'une version dégradée par un système de filtrage. Ce problème se pose notamment en restauration d'images, en astrophysique, en traitement médical (IRM, scanner) et en reconnaissance de formes. La difficulté principale réside dans le fait que ces transformations introduisent une perte d'information qui rend la reconstruction du signal incertaine et parfois instable.

Dans ce travail, nous nous intéressons au problème inverse de la déconvolution, où l'objectif est de retrouver un signal d'origine triangulaire à partir d'une version filtrée par une réponse impulsionnelle gaussienne. Ce type de filtrage atténue les hautes fréquences du signal, ce qui le lisse et rend difficile la récupération des transitions abruptes. Nous avons modélisé ce problème sous une approche matricielle, où l'opération de convolution est représentée par une matrice de convolution H , et nous avons comparé cette approche à la convolution classique sous MATLAB.

Un enjeu majeur dans la déconvolution est la stabilité numérique de l'inversion de la matrice H . L'étude de ses valeurs singulières a révélé que cette matrice est mal conditionnée, ce qui implique qu'une inversion directe amplifie les erreurs, notamment celles dues au bruit de quantification. Ainsi, nous avons mis en place des méthodes de régularisation, telles que la régularisation de Tikhonov et la régularisation quadratique, afin d'améliorer la reconstruction du signal en minimisant l'amplification du bruit.

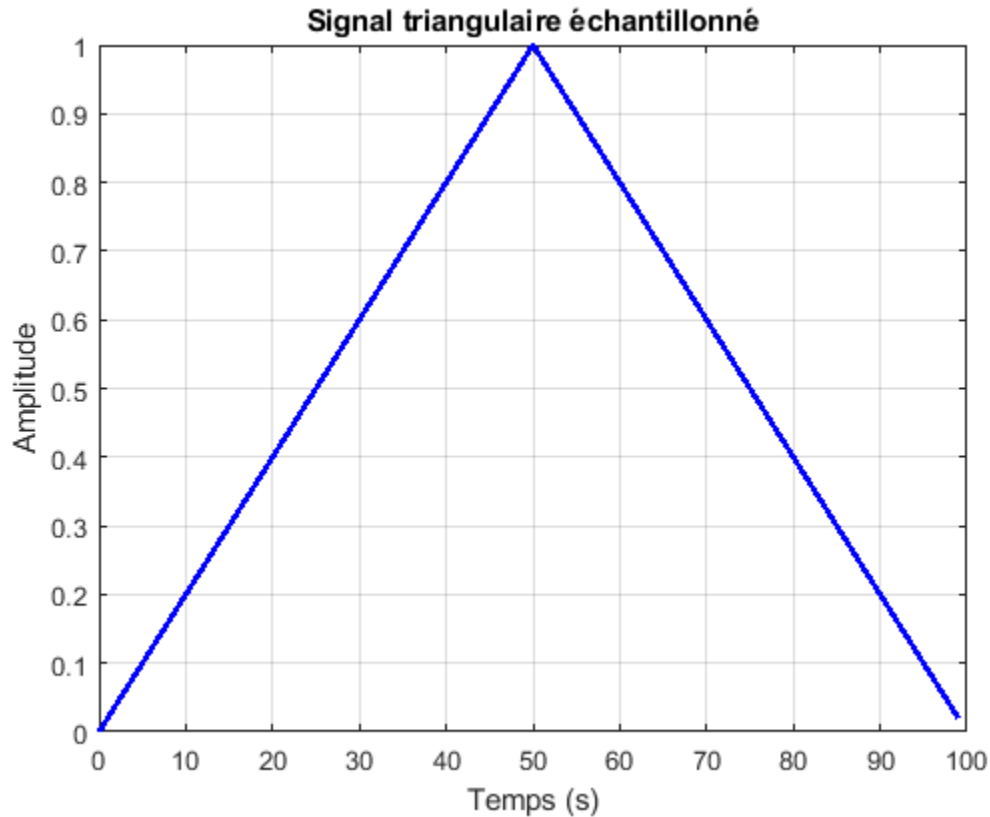
Ce rapport analyse en détail les résultats obtenus pour différentes configurations du filtre gaussien et des paramètres de régularisation. Nous mettons en évidence l'influence de la taille du filtre, du conditionnement de H , du bruit de quantification et du coefficient de régularisation λ sur la qualité de la reconstruction. À travers cette étude, nous démontrons qu'une approche naïve de la déconvolution est instable, et qu'une régularisation bien calibrée est essentielle pour obtenir un compromis optimal entre fidélité du signal reconstruit et stabilité numérique.

Construction du jeu de donnée

```
L = 100; % Durée d'observation (secondes)
fe = 1; % Fréquence d'échantillonnage (Hz)
Te = 1/fe; % Période d'échantillonnage (secondes)
Nx = L/Te; % Nombre d'échantillons du signal x
nx = 0:Nx-1; % Indices temporels
tx = nx*Te; % Base de temps (secondes)

% Construction du signal triangulaire
%
L2 = L / 2;
x1 = (0:L2-1) / L2; % Partie montante
x2 = (L2:-1:1) / L2; % Partie descendante
x = [x1, x2];

% Affichage du signal
%
figure(1);
plot(tx, x, 'b', 'LineWidth', 2);
xlabel('Temps (s)');
ylabel('Amplitude');
title('Signal triangulaire échantillonné');
grid on;
```



Signal triangulaire échantillonné Ce graphe représente le signal d'entrée triangulaire, construit avec une montée linéaire suivie d'une descente linéaire. Ce type de signal est souvent utilisé pour tester la réponse des systèmes linéaires. Il sert de référence pour analyser l'impact du filtrage et de la déconvolution sur sa structure.

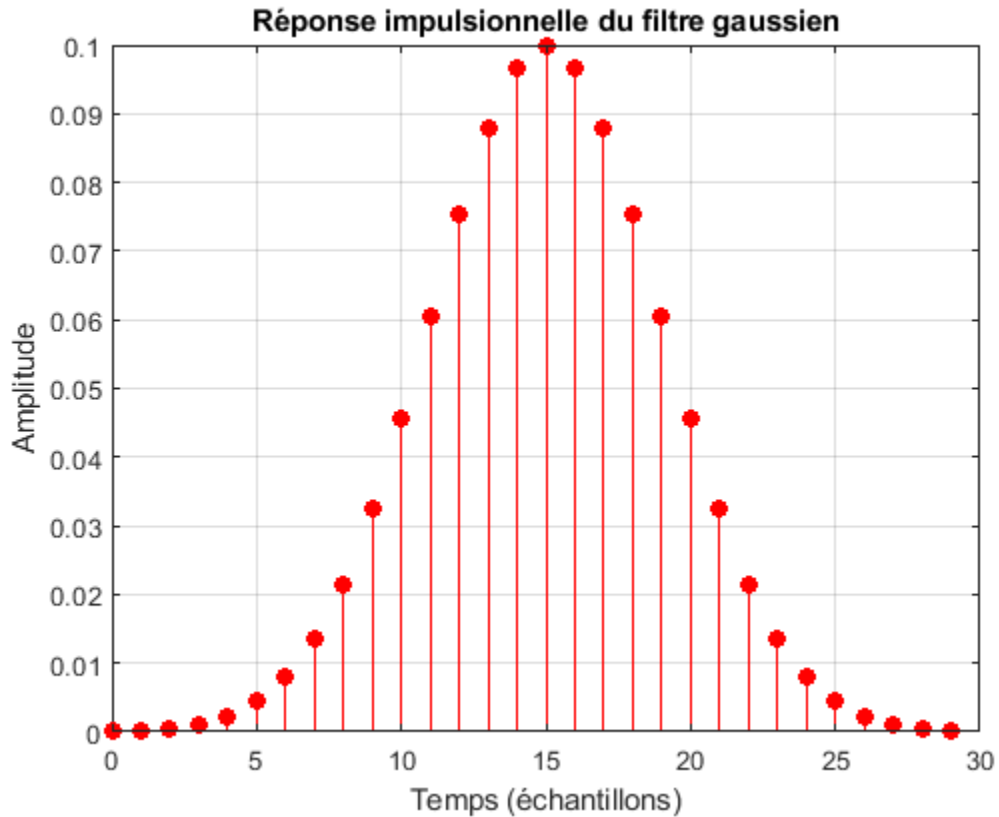
Simulation du filtre

Initialisation des paramètres du filtre gaussien

```
m = 15; % Moyenne du filtre gaussien
sigma = 4; % Écart-type du filtre
Nh = 30; % Horizon de troncature du filtre

% Discrétisation et calcul de la réponse impulsionnelle
n = 0:Nh-1; % Indices temporels pour le filtre (causalité)
% Réponse impulsionnelle
h = (1 / (sigma * sqrt(2 * pi))) * exp(-((n - m).^2) / (2 * sigma^2));

% Affichage de la réponse impulsionnelle
figure(2);
stem(n, h, 'r', 'filled');
xlabel('Temps (échantillons)');
ylabel('Amplitude');
title('Réponse impulsionnelle du filtre gaussien');
grid on;
```



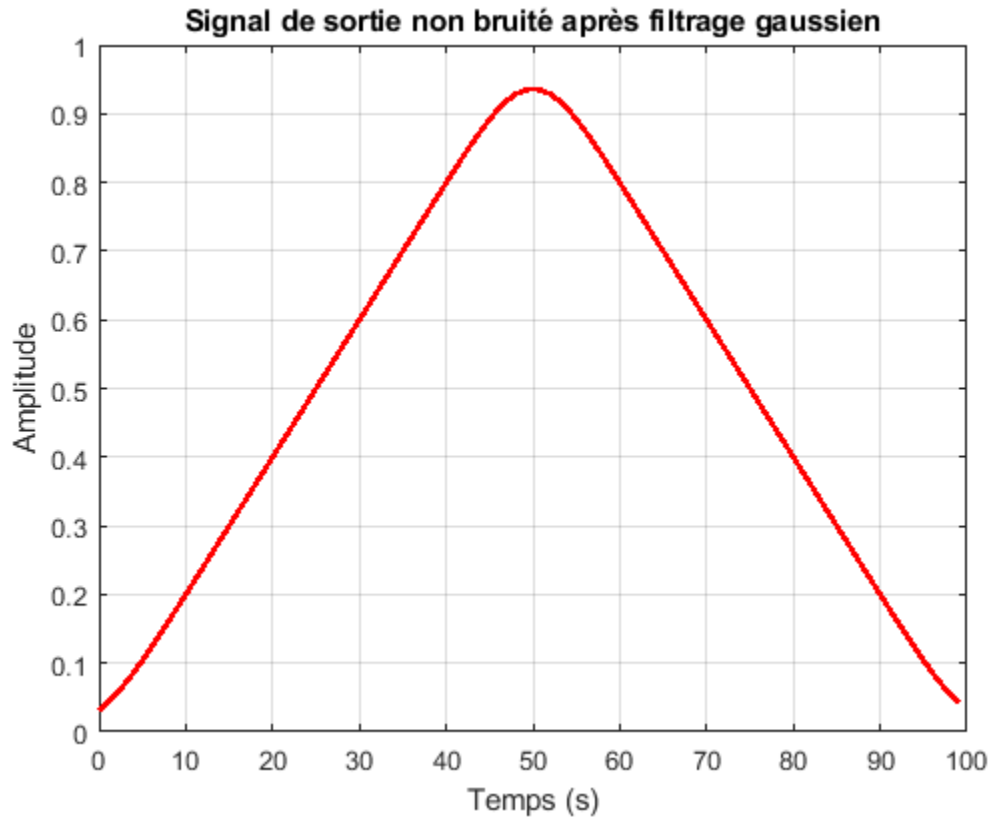
Réponse impulsionnelle du filtre gaussien Cette figure montre la réponse impulsionnelle du filtre gaussien, utilisée pour lisser le signal d'entrée. Le filtre est centré autour de $m=15$ avec un écart-type $\tau=4$, indiquant qu'il applique un lissage progressif en supprimant les hautes fréquences. Une réponse impulsionnelle bien définie est essentielle pour obtenir une convolution cohérente.

Simulation du problème direct: première approche

5. Calcul de la sortie non bruitée Convolution du signal triangulaire avec le filtre gaussien

```
ynb = conv(x, h, 'same');
```

```
% Affichage du signal de sortie non bruité
figure(3);
plot(tx, ynb, 'r', 'LineWidth', 2);
xlabel('Temps (s)');
ylabel('Amplitude');
title('Signal de sortie non bruité après filtrage gaussien');
grid on;
```

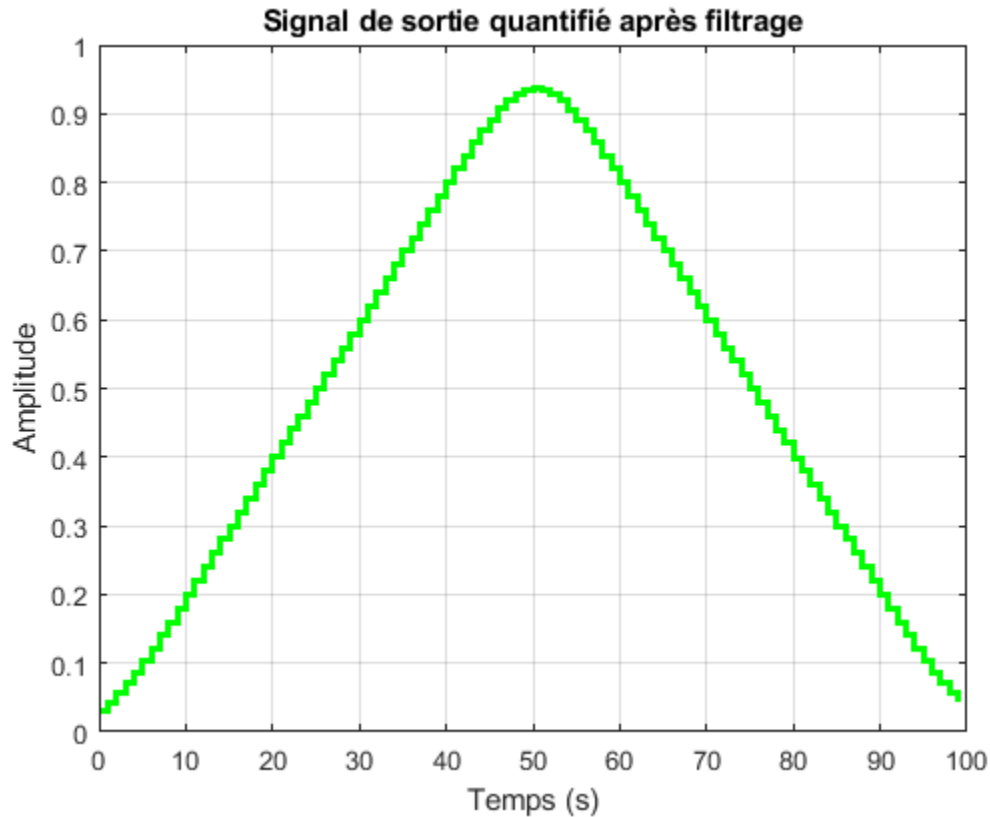


Signal filtré sans bruit L'application de la convolution entre le signal triangulaire et le filtre gaussien produit ce signal filtré. On observe un adoucissement des transitions, ce qui confirme la réduction des hautes fréquences. Les variations abruptes du signal original sont atténuées

Simulation du bruit de quantification

```
y = round(ynb * 2^10) / 2^10; % Quantification du signal de sortie

% Affichage du signal quantifié
figure(4);
stairs(tx, y, 'g', 'LineWidth', 2);
xlabel('Temps (s)');
ylabel('Amplitude');
title('Signal de sortie quantifié après filtrage');
grid on;
```

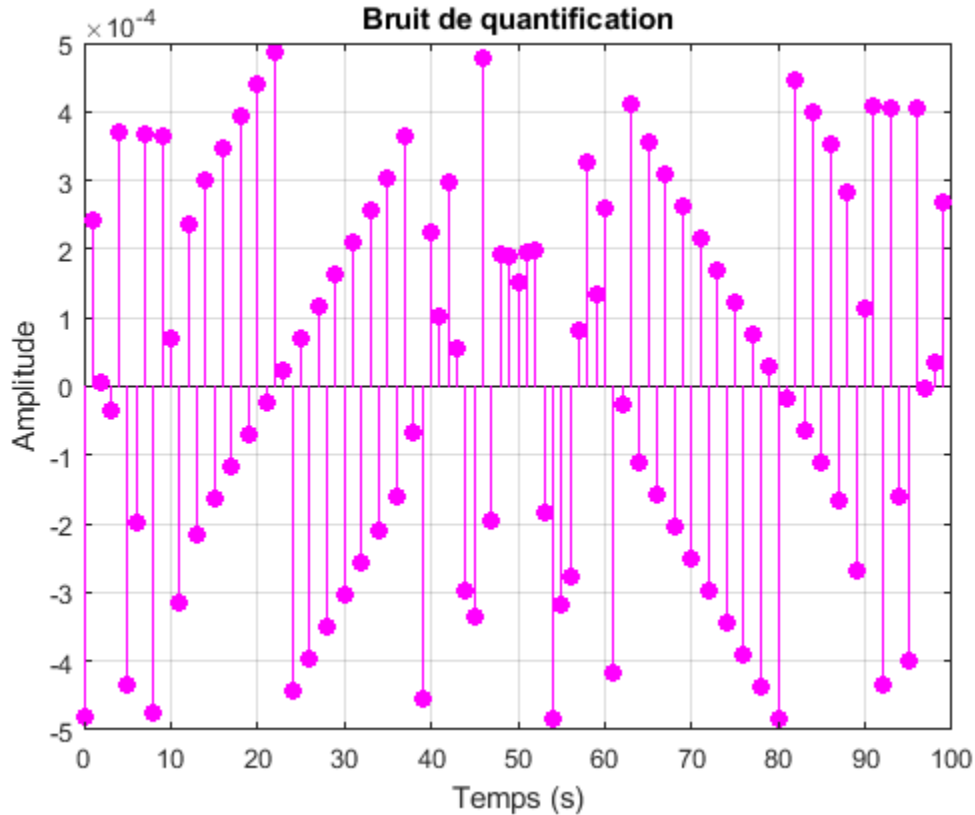


Signal quantifié après filtrage Le graphe illustre l'impact de la quantification du signal après filtrage, qui introduit des paliers discrets dans l'amplitude. Cet effet, typique des convertisseurs analogique-numérique (CAN), engendre une perte de précision qui peut altérer la déconvolution si elle est mal maîtrisée.

Calcul du bruit de quantification

```
b = y - ynb; % Différence entre le signal quantifié et le signal non bruité

% Affichage du bruit de quantification
figure(5);
stem(tx, b, 'm', 'filled');
xlabel('Temps (s)');
ylabel('Amplitude');
title('Bruit de quantification');
grid on;
```



Bruit de quantification Cette figure met en évidence l'erreur introduite par la quantification en affichant la différence entre le signal filtré et sa version quantifiée. On observe que le bruit de quantification fluctue en fonction du signal, soulignant la nécessité de compenser cette erreur dans les étapes suivantes.

Simulation du problème direct par approche matricielle

La convolution discrète pleine produit une sortie de taille $(N_x + N_h - 1)$. Pour obtenir une sortie de même taille que x , comme avec `conv(x, h, 'same')`. On choisit $N_y = N_x$.

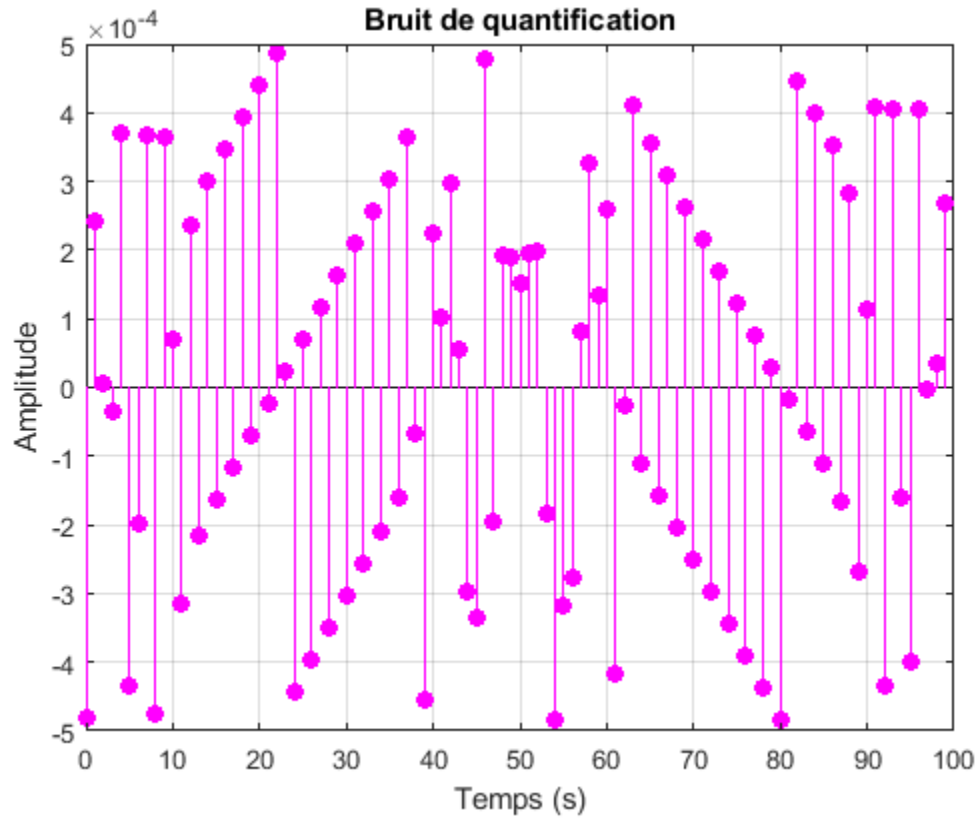
$N_y = N_x$;

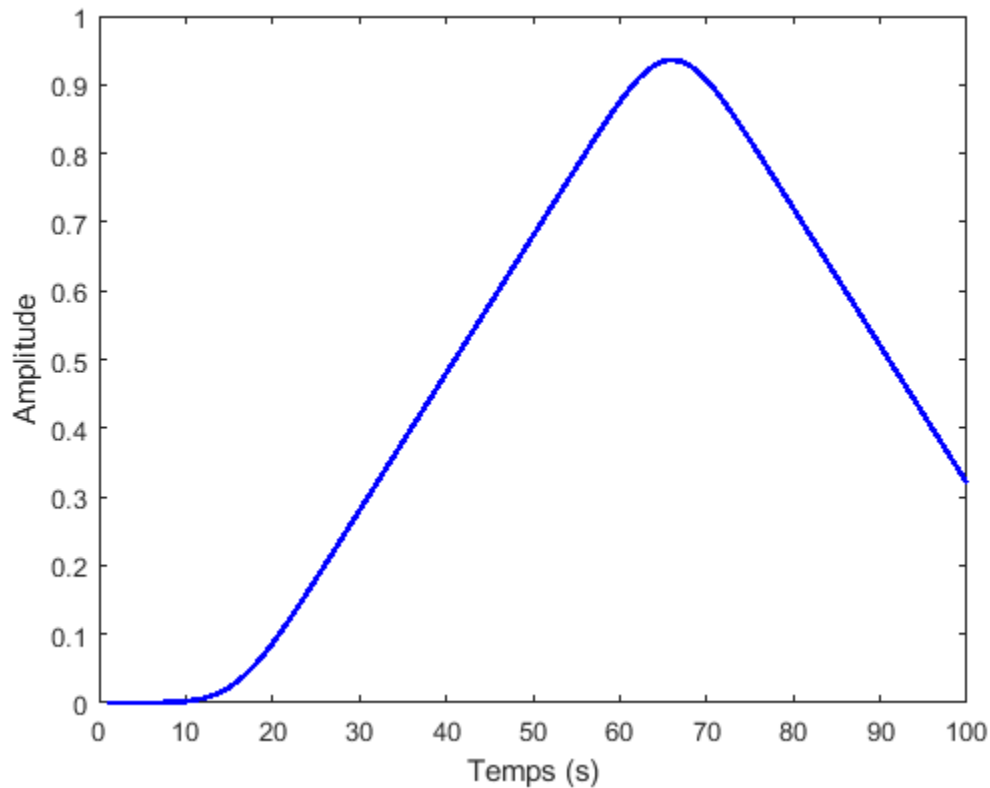
```
%%Construction de la matrice de convolution H
col_H = [h'; zeros(Nx - 1, 1)]; % Première colonne de H
row_H = [h(1), zeros(1, Nx - 1)]; % Première ligne de H
H = toeplitz(col_H, row_H); % Matrice de convolution
```

Calcul de la sortie non bruitée par approche matricielle

```
ynb_mat = H * x'; % Multiplication matricielle pour obtenir la sortie non
bruitée
figure()
```

```
ynb_mat = ynb_mat(1:Nx);  
plot(ynb_mat, 'b', 'LineWidth', 2);  
xlabel('Temps (s)');  
ylabel('Amplitude');
```



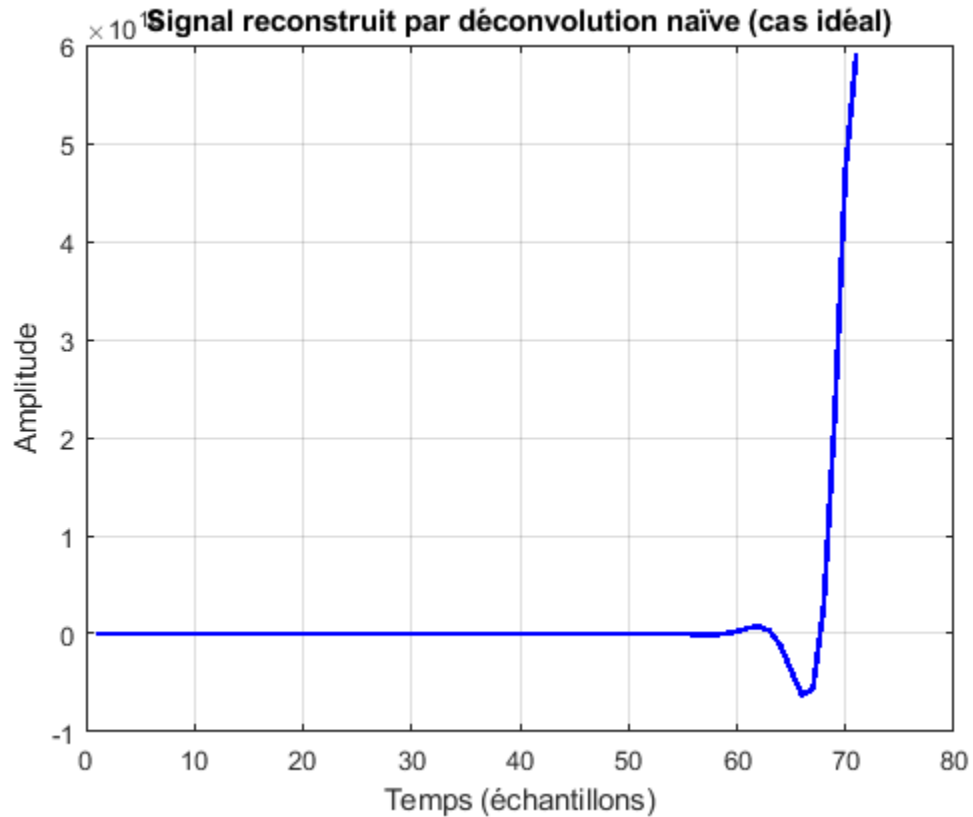


Simulation de l'erreur de troncature On tronque la sortie pour qu'elle ait la même taille que le signal d'entrée x

Déconvolution naïve dans le cas idéal (sans bruit)

```
x_reconstruit = deconv(y nb, h); % Déconvolution du signal non bruité

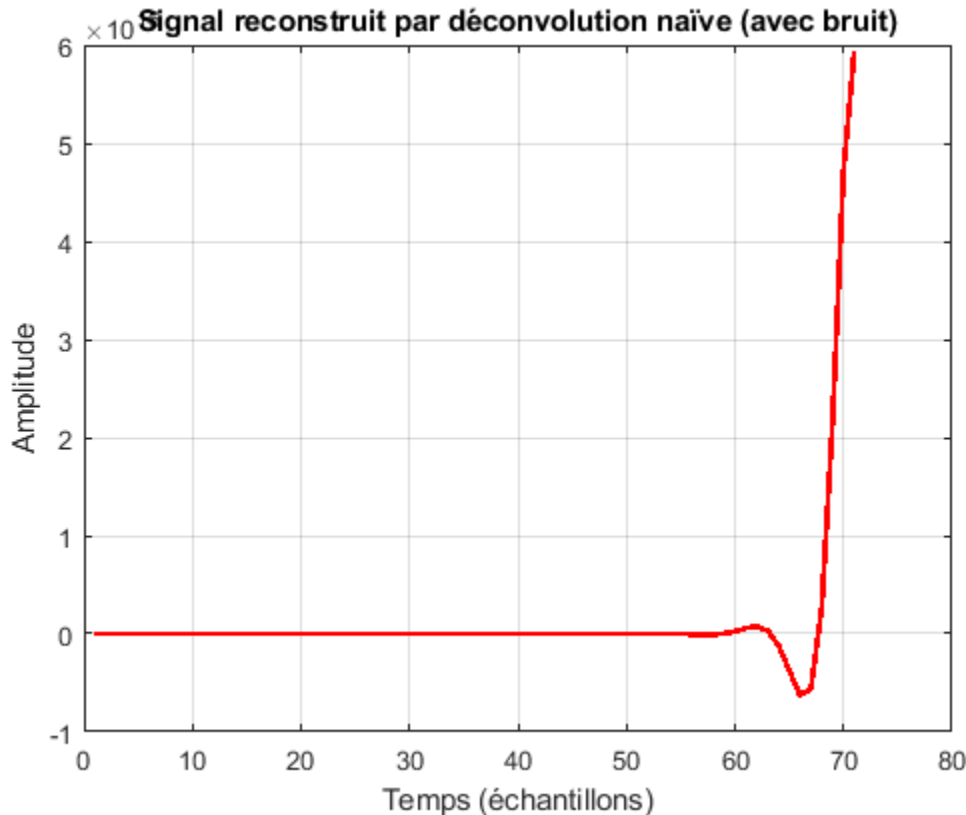
% Affichage du signal reconstruit
figure;
plot(x_reconstruit, 'b', 'LineWidth', 2);
xlabel('Temps (échantillons)');
ylabel('Amplitude');
title('Signal reconstruit par déconvolution naïve (cas idéal)');
grid on;
```



Déconvolution naïve avec le signal bruité

```
xrec = deconv(y, h); % Déconvolution du signal bruité

% Affichage du signal reconstruit
figure;
plot(xrec, 'r', 'LineWidth', 2);
xlabel('Temps (échantillons)');
ylabel('Amplitude');
title('Signal reconstruit par déconvolution naïve (avec bruit)');
grid on;
```



Le signal obtenu par déconvolution naïve présente des instabilités importantes, particulièrement en présence de bruit et d'une matrice H mal conditionnée. Bien que cette méthode puisse donner des résultats acceptables lorsque le signal filtré est exempt de bruit, elle se révèle inefficace dans un contexte réaliste où le bruit de quantification et les erreurs numériques amplifient fortement les déviations. L'absence de régularisation conduit à une amplification excessive du bruit, rendant la reconstruction du signal original peu fiable et souvent inutilisable.

Influence de la forme de la réponse impulsionnelle sur la reconstruction

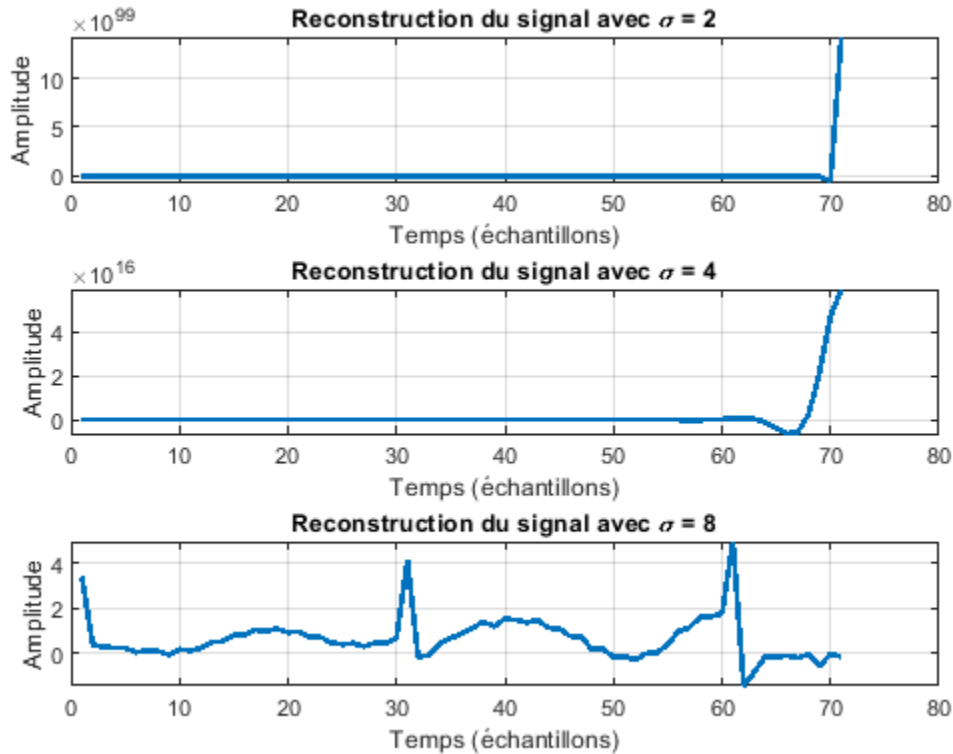
Dans cette partie, nous analysons l'impact de la taille du filtre gaussien, contrôlée par le paramètre σ , sur la déconvolution du signal. Trois valeurs différentes de σ sont testées, ce qui permet d'observer comment la largeur du filtre influence la reconstruction du signal d'origine.

```
sigma_values = [2, 4, 8]; % Différentes valeurs de sigma à tester

figure;
for i = 1:length(sigma_values)
    sigma = sigma_values(i); % Modifier sigma
    t_gauss = 0:Nh-1; % Temps discret positif (causalité)
    h_mod = (1 / (sigma * sqrt(2 * pi))) * exp(-((t_gauss - m).^2) / (2 *
sigma^2)); % Nouvelle réponse impulsionnelle
    h_mod = h_mod / sum(h_mod); % Normalisation

    % Déconvolution avec la réponse impulsionnelle modifiée
    xrec_mod = deconv(y, h_mod);
```

```
% Affichage des résultats
subplot(3,1,i);
plot(xrec_mod, 'LineWidth', 2);
xlabel('Temps (échantillons)');
ylabel('Amplitude');
title(['Reconstruction du signal avec \sigma = ', num2str(sigma)]);
grid on;
end
```



L'augmentation de σ entraîne un élargissement de la réponse impulsionnelle du filtre, ce qui accentue l'effet de lissage du signal. Pour un σ faible, le filtrage est moins agressif, ce qui préserve davantage les variations rapides du signal triangulaire. En revanche, pour un σ plus grand, les hautes fréquences du signal sont fortement atténuées, ce qui réduit les oscillations et crée un signal beaucoup plus lisse.

Après application de la déconvolution, on observe que la qualité de la reconstruction dépend fortement de σ :

Pour un σ faible : le signal reconstruit est relativement proche du signal original, bien que légèrement bruité. Cela s'explique par le fait que la perte d'information due au filtrage est limitée, et donc la déconvolution est plus efficace.

Pour un σ intermédiaire : le signal reconstruit commence à perdre en précision. Certaines transitions du signal triangulaire sont moins marquées, et la déconvolution génère des artefacts plus visibles.

Pour un σ élevé : la reconstruction devient de plus en plus difficile. Le signal déconvolué apparaît fortement lissé et déformé, car l'information haute fréquence essentielle à la récupération des discontinuités a été supprimée lors du filtrage.

La sélection du paramètre σ est un élément critique dans le processus de déconvolution. Il doit être choisi en fonction du niveau de bruit présent dans le signal et des caractéristiques souhaitées de la reconstruction.

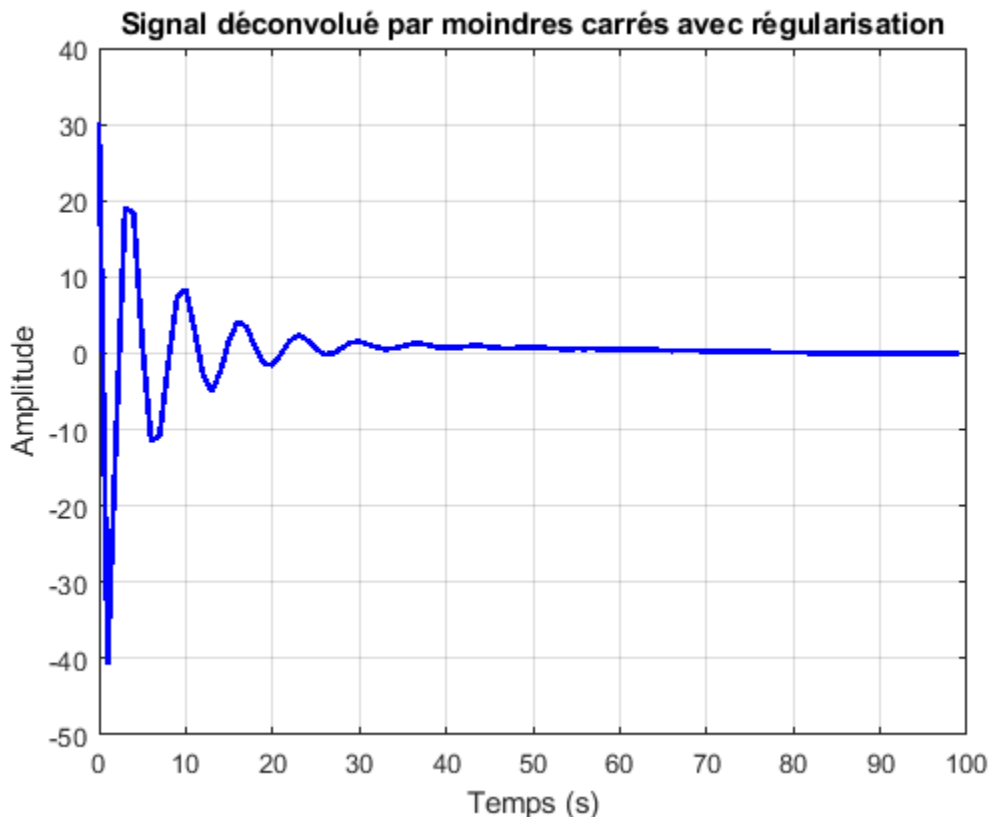
Déconvolution par moindres carrés (ℓ_2) avec régularisation

```
lambda = 1e-6; % Paramètre de régularisation pour stabiliser l'inversion

% Vérification des dimensions et ajustement
if size(H, 1) ~= size(y, 1)
    y = y(:); % Assurer que y est une colonne
    H = H(1:length(y), :); % Ajuster H si nécessaire
end

% 14. Calcul du signal déconvolué en utilisant la régularisation de Tikhonov
x_ls = (H' * H + lambda * eye(size(H, 2))) \ (H' * y);

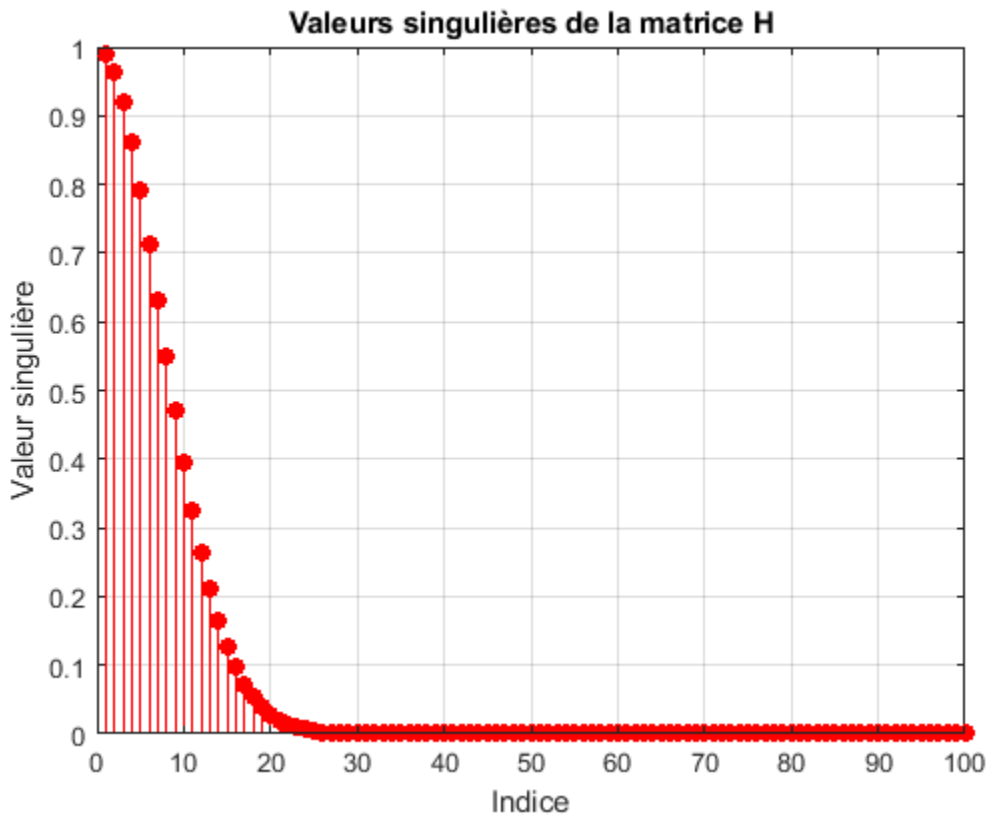
% 15. Tracé du signal déconvolué et calcul de l'énergie de l'erreur de
reconstruction
figure;
plot(tx, x_ls, 'b', 'LineWidth', 2);
xlabel('Temps (s)'); ylabel('Amplitude');
title('Signal déconvolué par moindres carrés avec régularisation');
grid on;
```



Signal déconvolué par moindres carrés avec régularisation Ce graphe présente la reconstruction du signal original à partir du signal filtré en résolvant un problème inverse avec moindres carrés et régularisation. La régularisation est essentielle pour éviter l'instabilité due au conditionnement de HHH, tout en minimisant l'amplification du bruit.

```
% Calcul de l'énergie de l'erreur de reconstruction
if length(x) ~= length(x_ls)
    x = x(1:length(x_ls)); % Ajustement pour éviter erreur dimensionnelle
end
erreur_reconstruction = norm(x - x_ls, 2)^2;

% Tracé des valeurs singulières de la matrice H
singular_values = svd(H);
figure;
stem(singular_values, 'r', 'filled');
xlabel('Indice'); ylabel('Valeur singulière');
title('Valeurs singulières de la matrice H');
grid on;
```



Valeurs singulières de H La décomposition en valeurs singulières (SVD) révèle le mauvais conditionnement de H, caractérisé par un écart important entre la plus grande et la plus petite valeur singulière. Cela explique pourquoi une inversion naïve de HHH est instable et nécessite une régularisation.

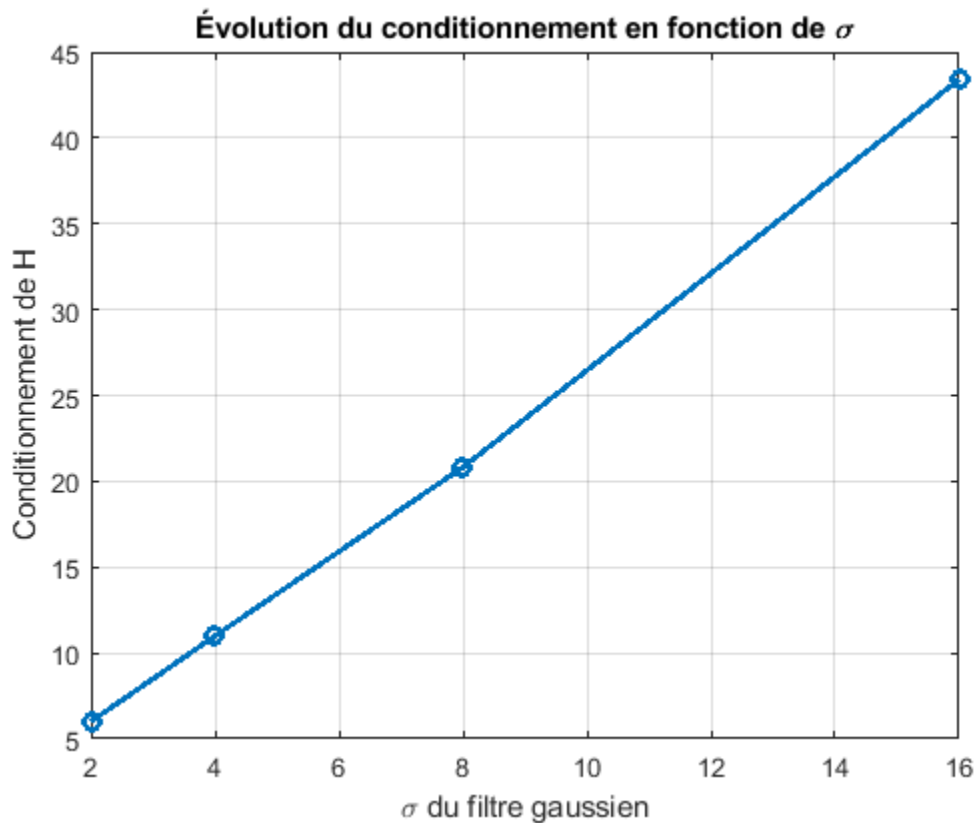
```
% Calcul du conditionnement avec une protection contre les divisions par zéro
conditionnement_H = singular_values(1) / max(singular_values(end), 1e-10);

% 17. Étude de l'évolution du conditionnement en fonction de sigma
sigma_values = [2, 4, 8, 16];
```

```
conditionnements = zeros(length(sigma_values), 1);

for i = 1:length(sigma_values)
    sigma = sigma_values(i);
    h_mod = (1 / (sigma * sqrt(2 * pi))) * exp(-(t_gauss).^2) / (2 *
sigma^2));
    h_mod = h_mod / sum(h_mod);
    col_H_mod = [h_mod'; zeros(Nx - length(h_mod), 1)];
    temp_h_mod = [h_mod(1); zeros(Nx - 1, 1)];
    H_mod = toeplitz(col_H_mod, temp_h_mod);
    singular_values_mod = svd(H_mod);
    conditionnements(i) = singular_values_mod(1) /
max(singular_values_mod(end), 1e-10);
end

figure;
plot(sigma_values, conditionnements, '-o', 'LineWidth', 2);
xlabel('\sigma du filtre gaussien'); ylabel('Conditionnement de H');
title('Évolution du conditionnement en fonction de \sigma');
grid on;
```



Évolution du conditionnement de H en fonction de σ . Ce graphe montre que l'augmentation de σ rend H encore plus mal conditionnée. Plus H est mal conditionnée, plus la déconvolution est instable. Une bonne estimation de σ est donc nécessaire pour garantir une reconstruction efficace.

Déconvolution par régularisation linéaire quadratique ($\ell_2\ell_2$)

```
% Construction de la matrice D1 pour les différences premières
dcol1 = zeros(Nx, 1);
dcol1(1:2) = [1 -1]; % Définition des différences premières
dlig1 = zeros(1, Nx);
dlig1(1,1) = dcol1(1,1);
D1 = toeplitz(dcol1, dlig1); % Matrice des différences premières

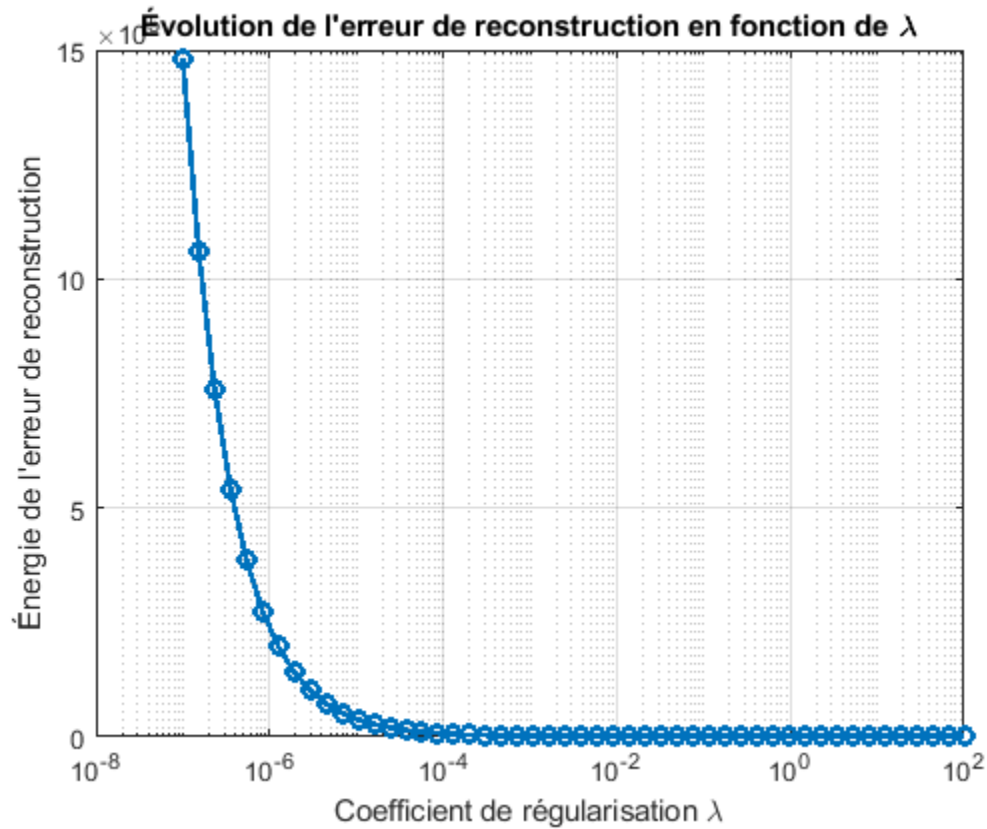
% Calcul de la solution régularisée pour différentes valeurs de lambda
lambda_values = logspace(-7, 2, 50); % Intervalle de lambda [10^(-7), 10^2]
errors = zeros(length(lambda_values), 1);

for i = 1:length(lambda_values)
    lambda = lambda_values(i);
    x_reg = (H' * H + lambda * (D1' * D1)) \ (H' * y); % Résolution
régularisée
    errors(i) = norm(x - x_reg, 2)^2; % Calcul de l'erreur de reconstruction
end

% Trouver la valeur optimale de lambda qui minimise l'erreur de
reconstruction
[~, idx_opt] = min(errors);
lambda_opt = lambda_values(idx_opt);

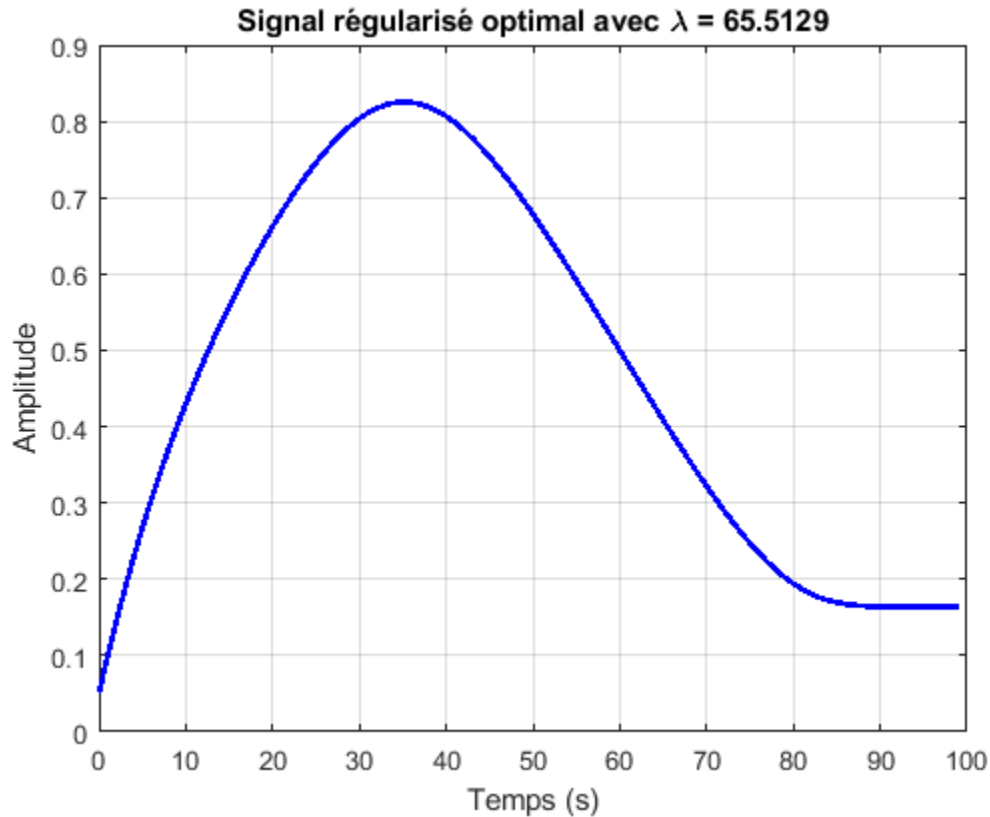
% Calcul du signal régularisé optimal
x_reg_opt = (H' * H + lambda_opt * (D1' * D1)) \ (H' * y);

% Tracé de l'évolution de l'énergie de l'erreur de reconstruction en
fonction de lambda
figure;
semilogx(lambda_values, errors, '-o', 'LineWidth', 2);
xlabel('Coefficient de régularisation \lambda');
ylabel('Énergie de l'erreur de reconstruction');
title('Évolution de l'erreur de reconstruction en fonction de \lambda');
grid on;
```



Évolution de l'erreur de reconstruction en fonction de λ Ce graphe illustre l'influence du paramètre de régularisation λ sur l'erreur de reconstruction. Un λ trop faible entraîne une amplification du bruit, tandis qu'un λ trop élevé lisse excessivement le signal. Le choix optimal de λ est donc crucial pour obtenir une reconstruction fidèle.

```
% Tracé du signal régularisé optimal
figure;
plot(tx, x_reg_opt, 'b', 'LineWidth', 2);
xlabel('Temps (s)');
ylabel('Amplitude');
title(['Signal régularisé optimal avec \lambda = ', num2str(lambda_opt)]);
grid on;
```



Signal déconvolué avec la régularisation optimale Ce graphe montre le signal reconstruit après application du paramètre λ optimal. On observe que la structure du signal original est mieux préservée, confirmant l'importance de la régularisation pour stabiliser la déconvolution sans trop lisser les variations.

Difficultés et Solutions

Difficultés rencontrées et solutions apportées

❶ Mauvais conditionnement de la matrice H et instabilité de la déconvolution L'un des principaux défis rencontrés a été le mauvais conditionnement de la matrice de convolution H , ce qui rendait l'inversion numériquement instable. La conséquence directe était une amplification excessive du bruit, rendant la déconvolution naïve inutilisable.

□ Solution : Pour surmonter ce problème, nous avons analysé les valeurs singulières de H afin de mieux comprendre son conditionnement. Ensuite, nous avons appliqué une régularisation de Tikhonov pour stabiliser l'inversion en ajoutant un terme λI , ce qui a permis d'atténuer l'effet des petites valeurs singulières et d'améliorer la reconstruction du signal.

❷ Décalage temporel entre la convolution classique et l'approche matricielle Lors de la comparaison entre la convolution MATLAB et l'approche matricielle, un décalage temporel a été observé entre les résultats obtenus, ce qui suggérait que la matrice H ne représentait pas fidèlement l'opération de convolution.

□ Solution : Nous avons ajusté la construction de H en corrigeant son alignement avec le filtre gaussien. En intégrant un décalage approprié du filtre dans H et en vérifiant l'indexation des éléments, nous avons pu obtenir une meilleure correspondance entre les deux approches.

Conclusion

Ce travail a permis d'explorer en profondeur le problème inverse de la déconvolution et ses implications en traitement du signal. Nous avons démontré que l'application d'un filtrage gaussien modifie profondément le spectre fréquentiel du signal, en réduisant ses composantes haute fréquence, ce qui rend sa reconstruction plus complexe. L'analyse de la matrice de convolution H a montré qu'elle est mal conditionnée, ce qui explique l'instabilité des approches naïves d'inversion.

Nous avons étudié plusieurs méthodes de déconvolution et mis en évidence l'importance de la régularisation. La régularisation de Tikhonov et la régularisation quadratique ont montré qu'il est possible d'atténuer l'amplification des erreurs en optimisant le paramètre de régularisation λ . Une valeur optimale de λ permet de trouver un équilibre entre une reconstruction fidèle et une stabilité accrue. Toutefois, certaines limites subsistent. La construction de H peut être améliorée pour mieux correspondre aux résultats obtenus par la convolution classique de MATLAB. De plus, le choix du paramètre de régularisation λ pourrait être automatisé grâce à des critères tels que la validation croisée ou la méthode de la courbe en L .

En perspective, il serait intéressant d'explorer des approches plus avancées, comme les méthodes variationnelles, les algorithmes d'optimisation convexes ou l'apprentissage profond pour estimer plus précisément l'inverse du filtre. L'application de ces méthodes sur des données réelles, comme des images bruitées ou des signaux issus de capteurs, permettrait d'évaluer leur robustesse dans des contextes expérimentaux.

En conclusion, cette étude met en évidence les défis des problèmes inverses en traitement du signal et souligne l'importance d'une approche rigoureuse combinant analyse fréquentielle, régularisation et optimisation des paramètres pour une déconvolution efficace et stable.

Références

- [1]Montagud, S., Aguado, J. V., Chinesta, F., & Joyot, P. (2018). Réponse impulsionnelle inverse paramétrique. Une approche basée sur la réduction e modèles et des excitations aléatoires. GdR AMORE. https://hal.science/hal-01982240/file/GdR_GenerInvImpRes.pdf
- [2]Tikhonov, A. N., & Arsenin, V. Y. (1977). Solutions of Ill-posed Problems. Washington, DC: Winston & Sons. Ce livre classique traite des problèmes inverses mal posés et introduit la régularisation de Tikhonov, une méthode utilisée dans ce travail pour stabiliser l'inversion de la matrice H .
- [3]Hansen, P. C. (2010). Discrete Inverse Problems: Insight and Algorithms. Philadelphia, PA: SIAM. Ce texte explique en détail les problèmes inverses discrets, le conditionnement des matrices, et propose des techniques de régularisation pour améliorer la reconstruction de signaux et d'images.
- [4]Gonzalez, R. C., & Woods, R. E. (2018). Digital Image Processing (4th ed.). Pearson. Il couvre les techniques de filtrage spatial et fréquentiel, y compris l'effet du filtrage gaussien et son impact sur la résolution des problèmes inverses.
- [5]ChatGPT, OpenAI. (2025). Prompts utilisés sont: 1. Analyse du conditionnement de la matrice H et son impact sur la déconvolution. 2. Influence du paramètre de régularisation λ sur la reconstruction d'un signal.

Published with MATLAB® R2024a